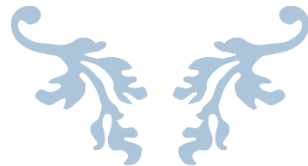




C PROGRAMMING LANGUAGE

INTERVIEW QUESTIONS & ANSWERS



Contents

1. When was C language developed?	15
2. Importance of variable declaration and initialization	15
3. Differentiate Source Codes from Object Codes	17
4. What is the significance of an algorithm to C programming?	17
5. Why is C language being considered a middle level language?	18
6. What are the different file extensions involved when programming in C?	18
7. Is it possible to create your own header files?	18
8. What is C language?	18
9. Who developed C language?	18
10. Describe about history of C programming language.....	19
11. Where is C programming language used or uses of C language?.....	20
12. What is the difference between C and C++?	21
Difference Between C and C++	21
13. What is the difference between top down approach and bottom up approach in programming languages?	23
14. What is the difference between C and Java?	24
15. C language has been developed in which language?	24
16. Which year C language is developed?	24
17. What is meant by programming language and give some examples?	25
18. Describe about C standards.....	25
19. What are the key features of C language or what are the characteristics of C language?	25
20. What is embedded C?	26
21. Which level is C language belonging to?	26
22. What do you mean by high level, middle level and low level languages and give an example for each?.....	26
23. What is the difference between structured oriented, object-oriented and non- structure oriented programming language?	27
24. What is a compiler?	27
25. What is the difference between assembler, compiler and interpreter?	28

26. What is meant by protocol?	28
27. List out some of C compilers.	28
28. Is C language case sensitive?	28
29. Why is C known as a mother language?	29
30. Why is C called a mid-level programming language?	29
31. Who is the founder of C language?	29
32. What are the features of the C language?	29
33. What is command line argument?	30
34. What is the acronym for ANSI?	30
35. Who is the main contributor in designing the C language after Dennis Ritchie?	31
36. What is a token?	31
37. What is the purpose of the keyword typedef?	31
38. What is difference between including the header file with-in angular braces < > and double quotes " "	32
39. What is keyword auto for?	32
40. What is volatile keyword?	32
41. What are main characteristics of C language?	33
42. How will you print "Hello World" without semicolon?	33
43. What is a static identifier?	33
44. What are the valid places for the keyword break to appear	33
45. Describe the history of C programming language	34
46. Which key word is used to perform unconditional branching?	35
47. Which compiler switch to be used for compiling the programs using math library with gcc compiler?	36
48. Can the curly brackets { } be used to enclose a single line of code?	36
49. What are header files and what are its uses in C programming?	36
50. What is syntax error?	36
51. Can two or more operators such as and be combined in a single line of program code?	37
52. Why is it that not all header files are declared in every C program?	37

53. When is the "void" keyword used in a function?	37
54. What are compound statements?	37
55. What is wrong in this statement? scanf("%d",whatnumber);	38
56. What are comments and how do you insert it in a C program?	38
57. What are reserved words?	38
58. Not all reserved words are written in lowercase. TRUE or FALSE?	38
59. What are run-time errors?	38
60. What is the difference between functions abs() and fabs()?	39
61. What is the general form of a C program?	39
62. What is the use of a semicolon (;) at the end of every program statement?	39
63. What is Standard Header Files ?	39
64. What is printf()?	40
65. What is scanf()?	40
66. Execution of a C program starts from which function?	40
67. What are all the sections that a C program may have and must have?	41
68. What is IDE?	41
69. What is header file in C language?	41
70. What is void in C?	42
71. What is token in C?	42
72. What are the types of C tokens?	43
73. What is identifier in C?	43
74. What is keyword in C?	43
75. List out some keywords available in C language.	43
76. What is the syntax for comments in C?	44
77. What is the use of printf() and scanf() functions?	44
78. Can we compile a program without main() function?	44
79. What is a token?	45
80. What is the difference between getch() and getche()?	45
81. What is the newline escape sequence?	47
List of Escape Sequences in C	47

82. What is the maximum length of an identifier?	47
83. Write a program to print "hello world" without using a semicolon?	48
84. How a negative integer is stored.....	48
85. Can I use "int" data type to store the value 32768? Why?.....	48
86. What are the different data types in C?	48
87. What is data type in C?.....	48
88. What is the difference between int, char, float and double data types?	49
89. What is modifier in C?	49
90. What are different types of modifiers in C?	49
91. What is typecasting?	50
92. What is the difference between variable declaration and variable definition? (in a line)	50
93. Where an automatic variable is stored?	50
94. When should we use the register storage specifier?	50
95. What is the purpose of extern storage specifier?	51
96. What is a static variable?	51
97. Can a variable be both const and volatile?	52
98. What are local static variables? What is their use?	52
99. What is the difference between declaration and definition of a variable/function	53
100. What are different storage class specifiers in C?	54
101. What is the difference between declaration and definition of a variable/function	54
102. What does static variable mean?.....	55
103. Where is the auto variables stored?	55
104. Where does global, static, and local, register variables, free memory and C Program instructions get stored?	56
105. Describe about storage allocation and scope of global, extern, static, local and register variables?	56
106. What are register variables? What are the advantages of using register variables?	57

107. What is storage class? What are the different storage classes in C?	57
108. What is variable initialization and why is it important?.....	57
109. What is wrong with this program statement? void = 10;	57
110. Is this program statement valid? INT = 10.50;	57
111. What are global variables and how do you declare them?	58
112. What is Blocks and Scope in c ?.....	58
113. What is Variable Storage Classes?	59
114. What is Variable Declaration ?.....	60
115. What is variable in C?.....	60
116. What is the difference between constant and variable in C?.....	60
117. Can variable name start with numbers?.....	61
118. What is the difference between variable declaration and variable definition in C?	61
119. What are the different types of variable in C?	61
120. What is local variable in C?	61
121. What is global variable in C?	Error! Bookmark not defined.
122. What is environment variable in C?	Error! Bookmark not defined.
123. What is the difference between auto variable and register variable in C?..	Error! Bookmark not defined.
124. What is the difference between auto variable and static variable in C?.....	Error! Bookmark not defined.
125. What is the difference between the local variable and global variable in C?	Error! Bookmark not defined.
126. What is the use of a static variable in C?	Error! Bookmark not defined.
127. What is static memory allocation?	Error! Bookmark not defined.
128. What is an auto keyword in C?	Error! Bookmark not defined.
129. What is lvalue and rvalue?	Error! Bookmark not defined.
130. What is l-value?	Error! Bookmark not defined.
131. Can we specify variable field width in a scanf() format string? If possible how?	Error! Bookmark not defined.
132. What are variables and it what way is it different from constants?	Error! Bookmark not defined.

133. What is constant in C? **Error! Bookmark not defined.**
134. What are the types of constants in C? **Error! Bookmark not defined.**
135. What is the difference between a variable and a constant in C programming?
..... **Error! Bookmark not defined.**
- What is the difference between a variable and a constant in C programming?
..... **Error! Bookmark not defined.**
136. S++ or S = S+1, which can be recommended to increment the value by 1 and why? **Error! Bookmark not defined.**
137. What is difference between i++ and ++i? **Error! Bookmark not defined.**
138. What are enumerations? **Error! Bookmark not defined.**
139. Where the address of operator (&) cannot be used for? **Error! Bookmark not defined.**
140. What is reminder for 5.0 % 2? **Error! Bookmark not defined.**
141. How many operators are there under the category of ternary operators? .. **Error! Bookmark not defined.**
142. Explain the use of comma operator (,). **Error! Bookmark not defined.**
143. How do you construct an increment statement or decrement statement in C?
..... **Error! Bookmark not defined.**
144. What is the difference between the = symbol and == symbol? **Error! Bookmark not defined.**
145. What is the modulus operator? **Error! Bookmark not defined.**
146. Which of the following operators is incorrect and why? (>=, <=, <>, ==) **Error! Bookmark not defined.**
147. What does the && operator do in a program code? **Error! Bookmark not defined.**
148. What is || operator and how does it function in a program? **Error! Bookmark not defined.**
149. Describe the order of precedence with regards to operators in C. **Error! Bookmark not defined.**
150. What is the difference between the expression "++a" and "a++"? **Error! Bookmark not defined.**
151. What are enumerated types? **Error! Bookmark not defined.**
152. What is the use of sizeof() function in C? **Error! Bookmark not defined.**
153. What is enum in C? **Error! Bookmark not defined.**
154. What is operator in C? **Error! Bookmark not defined.**

155. What are the different types of operator in C? **Error! Bookmark not defined.**
156. What is the syntax for ternary operator in C? **Error! Bookmark not defined.**
157. What is arithmetic operator in C? **Error! Bookmark not defined.**
158. What is assignment operator in C? **Error! Bookmark not defined.**
159. What is the relational operator in C? **Error! Bookmark not defined.**
160. What is the logical operator in C? **Error! Bookmark not defined.**
161. What is the bitwise operator in C? **Error! Bookmark not defined.**
162. What is the difference between pre increment operator and post increment operator? **Error! Bookmark not defined.**
163. What is the difference between pre decrement operator and post decrement operator? **Error! Bookmark not defined.**
164. What is & and * operators in C? **Error! Bookmark not defined.**
- Explanation **Error! Bookmark not defined.**
- Practical Example **Error! Bookmark not defined.**
165. What value will be assigned to the variable X if a = 10, b = 20, c = 30, d = 40 for the expression $X = a/b + c*d - c$? **Error! Bookmark not defined.**
166. What is the value assigned to the following variables? int X1 = 13/3; int X2 = 13%3; **Error! Bookmark not defined.**
167. Does a break is required by default case in switch statement? **Error! Bookmark not defined.**
168. In a switch statement, what will happen if a break statement is omitted? . **Error! Bookmark not defined.**
169. How will you print "Hello World" without semicolon in c programming Language? **Error! Bookmark not defined.**
170. How will you print "Hello World" without semicolon in c programming Language? **Error! Bookmark not defined.**
171. What are all decision control statements in C? **Error! Bookmark not defined.**
172. What will happen if break statement is not used in switch case in C? **Error! Bookmark not defined.**
- Example..... **Error! Bookmark not defined.**
173. Why is default statement used in switch case in C? **Error! Bookmark not defined.**
- Example..... **Error! Bookmark not defined.**

174. How will you print numbers from 1 to 100 without using a loop? **Error! Bookmark not defined.**

175. Differentiate between for loop and a while loop? What are its uses? **Error! Bookmark not defined.**

176. Explain the syntax for for loop..... **Error! Bookmark not defined.**

177. What is a nested loop? **Error! Bookmark not defined.**

178. Write a loop statement that will show the following output: 1 12 123 1234 12345
..... **Error! Bookmark not defined.**

179. What is an endless loop? **Error! Bookmark not defined.**

180. What are all loop control statements in C? **Error! Bookmark not defined.**

181. What is the difference between while and do-while loops in C? **Error! Bookmark not defined.**

182. What is an infinite loop? **Error! Bookmark not defined.**

183. What is the meaning of base address of the array? **Error! Bookmark not defined.**

184. How do you access the values within an array? **Error! Bookmark not defined.**

185. What is the advantage of an array over individual variables? **Error! Bookmark not defined.**

186. What are multidimensional arrays? **Error! Bookmark not defined.**

187. Describe how arrays can be passed to a user defined function ... **Error! Bookmark not defined.**

188. What is the right way to Initialize array? **Error! Bookmark not defined.**

189. Can array subscripts have negative value in C? **Error! Bookmark not defined.**

190. What is an array in C? **Error! Bookmark not defined.**

191. What is the difference between Strings and Arrays? **Error! Bookmark not defined.**

192. What is the use of a character? **Error! Bookmark not defined.**

193. How do you declare a variable that will hold string values? **Error! Bookmark not defined.**

194. What is wrong with this statement? myName = "Nurujjaman"; **Error! Bookmark not defined.**

195. How do you determine the length of a string value that was stored in a variable? **Error! Bookmark not defined.**

196. How do you convert strings to numbers in C? **Error! Bookmark not defined.**

197. What is the difference between memcpy() and memmove() functions in C?
..... **Error! Bookmark not defined.**

198. What is the difference between strcpy() and strncpy() functions in C? **Error! Bookmark not defined.**
199. What is the difference between array and string in C? **Error! Bookmark not defined.**
200. Explain the use of %i format specifier w.r.t scanf()..... **Error! Bookmark not defined.**
201. what is modular programming. **Error! Bookmark not defined.**
202. Does a built-in header file contains built-in function definition? . **Error! Bookmark not defined.**
203. What is the difference between actual and formal parameters? **Error! Bookmark not defined.**
204. Can a program be compiled without main() function?..... **Error! Bookmark not defined.**
205. What are static functions? What is their use? **Error! Bookmark not defined.**
206. In header files whether functions are declared or defined? **Error! Bookmark not defined.**
207. Out of fgets() and gets() which function is safe to use and why? **Error! Bookmark not defined.**
208. Difference between strdup and strcpy? **Error! Bookmark not defined.**
209. What is recursion? **Error! Bookmark not defined.**
210. How can you print a (backslash) using any of the printf() family of functions.
..... **Error! Bookmark not defined.**
211. What are the different ways of passing parameters to the functions? Which to use when? **Error! Bookmark not defined.**
212. What is a static function?..... **Error! Bookmark not defined.**
213. What is the difference between Call by Value and Call by Reference? **Error! Bookmark not defined.**
214. How do you generate random numbers in C? **Error! Bookmark not defined.**
215. What could possibly be the problem if a valid function name such as tolower() is being reported by the C compiler as undefined? **Error! Bookmark not defined.**
216. What are actual arguments? **Error! Bookmark not defined.**
217. What does the function toupper() do? **Error! Bookmark not defined.**
218. Is it possible to have a function as a parameter in another function? **Error! Bookmark not defined.**
219. What is the difference between functions getch() and getche()? **Error! Bookmark not defined.**

220. Can you pass an entire structure to functions?..... **Error! Bookmark not defined.**
221. What is gets() function? **Error! Bookmark not defined.**
222. How do you search data in a data file using random access method? **Error! Bookmark not defined.**
223. Is there a built-in function in C that can be used for sorting data?..... **Error! Bookmark not defined.**
224. What is function prototype?..... **Error! Bookmark not defined.**
- Code:..... **Error! Bookmark not defined.**
- Output:..... **Error! Bookmark not defined.**
225. Where should type cast function not be used in C?..... **Error! Bookmark not defined.**
226. How many arguments can be passed to a function in C? **Error! Bookmark not defined.**
227. What is static function in C?..... **Error! Bookmark not defined.**
228. If you want to execute C program even after main function is terminated, which function can be used? **Error! Bookmark not defined.**
229. Is it possible to call atexit() function more than once in a C program?..... **Error! Bookmark not defined.**
230. What is exit() function in C? **Error! Bookmark not defined.**
231. What is the difference between exit() and return() in C?..... **Error! Bookmark not defined.**
232. Is there any inbuilt library function in C to remove leading and trailing spaces from a string? How will you remove them in C?..... **Error! Bookmark not defined.**
233. What is the use of the function in C? **Error! Bookmark not defined.**
234. Example of call by value in C Program **Error! Bookmark not defined.**
235. What is the advantage of declaring void pointers? **Error! Bookmark not defined.**
236. What is a dangling pointer? in one line **Error! Bookmark not defined.**
237. What is a pointer on pointer?..... **Error! Bookmark not defined.**
238. What is Dangling pointer? **Error! Bookmark not defined.**
239. When should we use pointers in a C program?..... **Error! Bookmark not defined.**
240. What is a pointer? **Error! Bookmark not defined.**
241. What are the uses of a pointer? **Error! Bookmark not defined.**
242. Difference between pass by reference and pass by value? **Error! Bookmark not defined.**

243. What is a far pointer? Where we use it?..... **Error! Bookmark not defined.**
244. What is a huge pointer? **Error! Bookmark not defined.**
245. What is a normalized pointer, how do we normalize a pointer? ... **Error! Bookmark not defined.**
246. When to use -> (arrow) operator. **Error! Bookmark not defined.**
247. What is a pointer to a function? Give the general syntax for the same. **Error! Bookmark not defined.**
248. What is a NULL statement? **Error! Bookmark not defined.**
249. What are pointers? **Error! Bookmark not defined.**
250. What is null pointer in C? **Error! Bookmark not defined.**
251. What is NULL in C? **Error! Bookmark not defined.**
252. What is void pointer in C? **Error! Bookmark not defined.**
253. What is dangling pointer in C?..... **Error! Bookmark not defined.**
254. What is wild pointer in C? **Error! Bookmark not defined.**
255. What is file pointer in C?..... **Error! Bookmark not defined.**
256. When can void pointer and null pointer be used in C? **Error! Bookmark not defined.**
257. What is const pointer in C?..... **Error! Bookmark not defined.**
258. What is the difference between call by value and call by reference in C? . **Error! Bookmark not defined.**
259. Example of call by reference in C Program **Error! Bookmark not defined.**
260. What is a pointer in C?..... **Error! Bookmark not defined.**
261. What is the usage of the pointer in C? **Error! Bookmark not defined.**
262. What is a NULL pointer and far pointer in C? **Error! Bookmark not defined.**
263. What is dangling pointer in C?..... **Error! Bookmark not defined.**
264. What is pointer to pointer in C? **Error! Bookmark not defined.**
265. What is the difference between near, far and huge pointers? **Error! Bookmark not defined.**
266. Can we access the array using a pointer in C language? **Error! Bookmark not defined.**
267. What is a self-referential structure?..... **Error! Bookmark not defined.**
268. What is a nested structure? **Error! Bookmark not defined.**
269. What is a structure?..... **Error! Bookmark not defined.**

270. What are the differences between structures and arrays? **Error! Bookmark not defined.**
271. What is the use of typedef? **Error! Bookmark not defined.**
272. What are bit fields? **Error! Bookmark not defined.**
273. What are structure types in C? **Error! Bookmark not defined.**
274. What is the structure? **Error! Bookmark not defined.**
275. What is a union? **Error! Bookmark not defined.**
276. What are the differences between structures and union? **Error! Bookmark not defined.**
277. What the advantages of using Unions? **Error! Bookmark not defined.**
278. What is a union? **Error! Bookmark not defined.**
279. Explain the purpose of the function `sprintf()` **Error! Bookmark not defined.**
280. Describe the file opening mode "w+" **Error! Bookmark not defined.**
281. Is FILE a built-in data type? **Error! Bookmark not defined.**
282. What is a sequential access file? **Error! Bookmark not defined.**
283. What does the characters "r" and "w" mean when writing programs that will make use of files? **Error! Bookmark not defined.**
284. What is the difference between text files and binary files? **Error! Bookmark not defined.**
285. What is the advantage of a random access file? **Error! Bookmark not defined.**
286. Are comments included during the compilation stage and placed in the EXE file as well? **Error! Bookmark not defined.**
287. What is the purpose of `sprintf()` function? **Error! Bookmark not defined.**
288. What are the functions to open and close the file in C language? **Error! Bookmark not defined.**
289. Distinguish between `malloc()` and `calloc()` memory allocation. . **Error! Bookmark not defined.**
290. What is memory leak? Why it should be avoided **Error! Bookmark not defined.**
291. What are the differences between `malloc ()` and `calloc ()`? **Error! Bookmark not defined.**
292. What are macros? What are its advantages and disadvantages? **Error! Bookmark not defined.**
293. Difference between arrays and linked list? **Error! Bookmark not defined.**

294. What is dynamic memory allocation?..... **Error! Bookmark not defined.**
295. What functions are used for dynamic memory allocation in C language? **Error! Bookmark not defined.**
296. What is the difference between malloc() and calloc()? **Error! Bookmark not defined.**
297. What is a preprocessor? **Error! Bookmark not defined.**
298. How will you print "Hello World" without semicolon in c programming Language using #define? **Error! Bookmark not defined.**
- code **Error! Bookmark not defined.**
- code **Error! Bookmark not defined.**
299. What is Macro? Why do we use the macro?..... **Error! Bookmark not defined.**
300. What is the use of #define in C? **Error! Bookmark not defined.**
301. What is ## operator in C?..... **Error! Bookmark not defined.**
302. What is pragma in C? Or how will you execute functions before and after main function in C program? **Error! Bookmark not defined.**
303. How will you override an existing macro in C? **Error! Bookmark not defined.**
304. How to check whether macro is defined or not in a C program? **Error! Bookmark not defined.**
305. What are the different types of control structures in programming?..... **Error! Bookmark not defined.**
306. What is the use of goto statement?..... **Error! Bookmark not defined.**
307. What is recursion in C? **Error! Bookmark not defined.**
308. What are command line arguments? **Error! Bookmark not defined.**
309. What is the purpose of built-in strcmp() function..... **Error! Bookmark not defined.**
310. Which function in C can be used to append a string to another string?..... **Error! Bookmark not defined.**
311. What is the difference between memcpy() and strcpy() functions in C?..... **Error! Bookmark not defined.**

1. When was C language developed?

C language was developed in 1972 at bell laboratories of AT&T.

More Information

C, computer programming language developed in the early 1970s by American computer scientist Dennis M. Ritchie at Bell Laboratories (formerly AT&T Bell Laboratories). C was designed as a minimalist language to be used in writing operating systems for minicomputers, such as the DEC PDP 7, which had very limited memories compared with the mainframe computers of the period. The language was devised during 1969–73, alongside the early development of the UNIX operating system. It was based on CPL (Combined Programming Language), which had been first condensed into the B programming language—a stripped-down computer programming language—created in 1969–70 by Ken Thompson, an American computer scientist and a colleague of Ritchie. Ritchie subsequently rewrote and restored features from CPL to create C and eventually rewrote the UNIX operating system in the new language.

2. Importance of variable declaration and initialization

1. Variable declaration in Java is the process of informing the compiler about the variable name, type, and size.

2. Initialization is the process of giving a variable an initial value.
3. Variable declaration and initialization are important in Java because they help to prevent errors and improve code readability.
4. Declaring a variable before using it ensures that the variable has a known type and size, which can help the compiler optimize memory usage and improve performance.
5. Initializing a variable before using it ensures that the variable has a valid value, which can help to prevent errors caused by using uninitialized variables.
6. Proper variable declaration and initialization can make it easier to debug your code and understand how it works.
7. It also improves the readability and maintainability of your code.
8. Proper variable declaration and initialization are essential for good programming practice and are commonly used in many programming languages, including Java.

In conclusion, variable declaration and initialization in the Java programming language is an important concept that ensures the proper functioning of a

program. Declaring variables before use ensures that they are set to a specific type, and assigning an initial value to a variable ensures that it has a starting value that can be used in computations or other operations. Not declaring and initializing variables can lead to errors in the program, such as null pointer exceptions or unexpected outputs. It is important to properly declare and initialize variables to ensure that a program runs smoothly and produces the expected results.

3. Differentiate Source Codes from Object Codes

Source codes are codes that were written by the programmer. It is made up of the commands and other English-like keywords that are supposed to instruct the computer what to do. However, computers would not be able to understand source codes. Therefore, source codes are compiled using a compiler. The resulting outputs are object codes, which are in a format that can be understood by the computer processor. In C programming, source codes are saved with the file extension .C, while object codes are saved with the file extension .OBJ

4. What is the significance of an algorithm to C programming?

Before a program can be written, an algorithm has to be created first. An algorithm provides a step by step procedure on how a solution can be derived. It also acts as a blueprint on how a program will start and end, including what process and computations are involved.

5. Why is C language being considered a middle level language?

This is because C language is rich in features that make it behave like a high level language while at the same time can interact with hardware using low level methods. The use of a well structured approach to programming, coupled with English-like words used in functions, makes it act as a high level language. On the other hand, C can directly access memory structures similar to assembly language routines.

6. What are the different file extensions involved when programming in C?

Source codes in C are saved with .C file extension. Header files or library files have the .H file extension. Every time a program source code is successfully compiled, it creates an .OBJ object file, and an executable .EXE file.

7. Is it possible to create your own header files?

Yes, it is possible to create a customized header file. Just include in it the function prototypes that you want to use in your program, and use the #include directive followed by the name of your header file.

8. What is C language?

- C language is a structure/procedure oriented, middle level programming language developed at Bell Laboratories in 1972 by Dennis Ritchie.
 - C language was invented for implementing UNIX operating system.
 - In 1978, Dennis Ritchie and Brian Kernighan published the first edition **The C Programming Language**.
 - Also, C language is an ANSI/ISO standard and powerful programming language for developing real time applications
-

9. Who developed C language?

C, computer programming language developed in the early 1970s by American computer scientist **Dennis M. Ritchie** at **Bell Laboratories (formerly AT&T Bell Laboratories)**. C was designed as a minimalist language to be used in writing operating systems for minicomputers, such as the DEC PDP 7, which had very limited memories compared with the mainframe computers of the period. The language was devised during 1969–73, alongside the early development of the UNIX operating system. It was based on CPL (Combined Programming Language), which had been first condensed into the B programming language—a stripped-down computer programming language—created in 1969–70 by Ken Thompson, an American computer scientist and a colleague of Ritchie. Ritchie subsequently rewrote and restored features from CPL to create C and eventually rewrote the UNIX operating system in the new language.

10. Describe about history of C programming language.

- C programming language is a structure oriented programming language, was developed at Bell Laboratories in 1972 by Dennis Ritchie
- C language features were derived from earlier language called “B” (Basic Combined Programming Language – BCPL)
- In earlier days, programs were written in assembly level language. So, it had happened to write very big programs to perform specific tasks using assembly code.
- But, ‘B’ language could perform the same task in few lines of program and it was faster than assemble language code.
- But, B language did not support some features like data types and structures etc. So, this was a drawback of B language. So, Dennis Ritchie developed C language by keeping most part of the B language and adding many features that produced powerful and effective outputs.
- So, C language was invented for implementing UNIX operating system. Most of the UNIX components were rewritten in C.
- In 1978, Dennis Ritchie and Brian Kernighan published the first edition “The C Programming Language” and commonly known as K&R C

- In 1983, the American National Standards Institute (ANSI) established a committee to provide a modern, comprehensive definition of C. The resulting definition, the ANSI standard, or “ANSI C”, was completed late 1988.
- C89/C90 standard – First standardized specification for C language was developed by American National Standards Institute in 1989. C89 and C90 standards refer to the same programming language.
- C99 standard – Next revision was published in 1999 that introduced new futures like advanced data types and other changes.
- C11 standard adds new features to C and library like type generic macros, anonymous structures, improved Unicode support, atomic operations, multi-threading, and bounds-checked functions. It also makes some portions of the existing C99 library optional, and improves compatibility with C++.
- Embedded C includes features not available in normal C like fixed-point arithmetic, named address spaces, and basic I/O hardware addressing.
-

11. Where is C programming language used or uses of C language?

- C language is used to develop system applications that forms major portion of operating systems such as Windows, UNIX and Linux. Operating systems, C compiler and all UNIX application programs are written in C language. Below are some examples of uses of C language.
 - Database systems
 - Graphics packages
 - Word processors
 - Spread sheets
 - Operating system development
 - Compilers and Assemblers

- Network drivers
- Interpreters

12. What is the difference between C and C++?

- Even though C and C++ programming languages are belonging to middle level languages, both are differed in below.
 - C is structure/procedure oriented programming language whereas C++ is object oriented programming language.
 - C language program design is top down approach whereas C++ is using bottom up approach.
 - Polymorphism, virtual function, inheritance, Operator overloading, namespace concepts are not available in C programming language. Whereas C++ language supports all these concepts and features.
 - C language gives importance to functions rather than data. Whereas C++ gives importance to data rather than functions.
 - So, data and function mapping is difficult in C. But, data and function mapping is simple in C++ that can be done using objects.
 - C language does not support user define data types. Whereas C++ supports user define data types.
 - Exception handling is not present in C programming language. Whereas exception handling is present in C++ language.
 - C language allows data to freely flow around the functions. But, data and functions are bound together in C++ which does not allow data to freely flow around the functions.

Difference Between C and C++

Parameter	C	C++
Definition	It is a structural programming language that doesn't provide any support for classes and objects.	It is an object-oriented programming language, and it provides support for the concept of classes and objects.

History	Dennis Ritchie developed the C language at the AT&T Bell Laboratories in around 1969.	Bjarne Stroustrup developed the C++ language in 1979-1980 at Bell Labs.
Type of Programming Language	C primarily supports procedural programming for developing codes. Here, it checks the code line by line.	C++ supports both programming paradigms- procedural as well as object-oriented. It is, thus, known as a hybrid language.
Support for OOPs Feature	C has no support for the OOPs concept. Thus, it does not support encapsulation, polymorphism, and inheritance.	The C++ language supports encapsulation, inheritance, and polymorphism because it is an object-oriented programming language.
Supported Features	C has no support for functions and operator overloading. It also does not have any namespace feature and functionality of reference variables.	C++, on the other hand, supports both of the functions and operator overloading. It also has the namespace feature and the functionality of reference variables.
Driven Type	The C is a function-driven language because it is procedural programming.	The C++ language, on the other hand, is object-driven because it is OOP (object-oriented programming).
Data Security	C is vulnerable to manipulation via outside code. It is because it does not support encapsulation- leading to its data behaving as a free entity.	C++, on the other hand, is a very secure language. It supports encapsulation of data in the form of objects- thus hiding the information and ensuring that one uses the structures and operators as intended.
Type of Subset	It is a subset of the C++ language. It cannot run the codes used in C++.	It is a superset of the C language. It is capable of running 99% of the C language codes.
Segregation of Data and Functions	Since C is a procedural programming language, the data and functions stay separate in it.	In the case of C++, the data and functions stay encapsulated in an object's form.
Hiding Data and Information	C does not support the hiding of data and information.	C++ language hides the data through encapsulation. This process ensures that a user utilizes the structures as operators as intended.
Built-in Data Types	The C language does not support built-in data types.	The C++ language supports built-in data types.
Function Inside Structures	In the case of C, it does not define the functions inside structures.	In the case of C++, it uses functions inside a structure.
Reference Variables	It does not support any reference variables.	It supports reference variables.
Overloading of Functions	Function overloading allows a user to have more than one function with different parameters but the same name. The C language does not support it.	The C++ language supports function overloading.
Overriding of Functions	Function overriding provides the specific implementation to any	The C++ language supports function overriding.

	function that is defined already in the base class. The C language does not support it.	
Header File	C language uses the <stdio.h> header file.	C++ language uses the <iostream.h> header file.
Namespace Features	The namespace feature groups various entities like objects, classes, and functions under a specific name. No namespace features are present in the C language.	The C++ language uses the namespace features to help avoid name collisions.
Virtual and Friend Functions	The C language does not support virtual and friend functions.	The C++ language supports virtual and friend functions.
Primary Focus	C language focuses on the process or method instead of focusing on the data.	C++ language focuses on the data instead of focusing on the procedure or method.
Inheritance	The inheritance feature assists the child class in reusing the parent class's properties. The C language offers no support for inheritance.	The C++ language provides support for inheritance.
Allocation and Deallocation of Memory	The C language provides <code>calloc()</code> and <code>malloc()</code> for dynamic allocation of memory and <code>free()</code> for deallocation of memory.	The C++ language provides a new operator for the allocation of memory and a delete operator for the deallocation of memory.
Exception Handling	It does not provide any direct support for exceptional handling. C language requires the usage of functions that support exception handling.	It provides direct support for exceptional handling. The C++ language uses a try-catch block.
Access Modifiers	The structures in C have no access modifiers.	The structures in C++ do have access modifiers.
Type of Approach	C language follows a top-down approach. It functions to break down the main module into various tasks. Then it breaks these tasks into sub-tasks, and so on.	C++ language follows the bottom-up approach. It means that it first develops the lower-level modules and then moves on to the next-level modules.
Function for Input/Output	The C language uses the <code>scanf()</code> and <code>printf()</code> functions for the input and output operations.	In the C++ language, it uses the <code>cin</code> and <code>cout</code> for the input and output operations.

13. What is the difference between top down approach and bottom up approach in programming languages?

Top down approach and bottom up approach are involved in software development. These approaches are not involved in program execution. Structure/procedure oriented programming languages like C programming language follows top down approach. Whereas object oriented programming languages like C++ and Java programming language follows bottom up approach. Top down approach begins with high level design and ends with low level design or development. Whereas, bottom up approach begins with low level design or development and ends with high level design. In top down approach, main() function is written first and all sub functions are called from main function. Then, sub functions are written based on the requirement. Whereas, in bottom up approach, code is developed for modules and then these modules are integrated with main() function. Now-a-days, both approaches are combined together and followed in modern software design.

14. What is the difference between C and Java?

- C is structure/procedure oriented programming language whereas Java is object oriented programming language.
 - C language program design is top down approach whereas Java is using bottom up approach.
 - C language is middle level language whereas Java is high level language.
 - Exception handling is not present in C programming language. Whereas exception handling is present in Java.
 - Polymorphism, virtual function, inheritance, Operator overloading, namespace concepts are not available in C programming language. Whereas Java supports all these concepts and features.
-

15. C language has been developed in which language?

C language has been developed using assembly level language.

16. Which year C language is developed?

C programming language was developed at Bell Laboratories in 1972 by Dennis Ritchie.

17. What is meant by programming language and give some examples?

- A programming language is nothing but a language designed to communicate to machines through instructions and commands.
 - Normally machines are computers. Programs are written using some programming languages to control the behaviour of machines/computers and to make them to perform required tasks.
 - Programming language example: Assembly language, C language, C++ language, Java, C#, .NET, Python etc.
-

18. Describe about C standards.

- C89/C90 standard – First standardized specification for C language was developed by American National Standards Institute in 1989. C89 and C90 standards refer to the same programming language.
 - C99 standard – Next revision was published in 1999 that introduced new features like advanced data types and other changes.
 - C11 standard adds new features to C and library like type generic macros, anonymous structures, improved Unicode support, atomic operations, multi-threading, and bounds-checked functions. It also makes some portions of the existing C99 library optional, and improves compatibility with C++.
-

19. What are the key features of C language or what are the characteristics of C language?

key features of C language

- Reliability
- Portability
- Flexibility
- Interactivity
- Modularity

- Efficiency and Effectiveness
-

20. What is embedded C?

- Embedded C is the extension of C programming language.
 - Embedded C is used to develop micro controller based applications.
 - Embedded C includes features not available in normal C like fixed-point arithmetic, named address spaces, and basic I/O hardware addressing.
 - Cell phones, MP3 players are some example for embedded systems in which embedded C is used to program and control these devices.
-

21. Which level is C language belonging to?

- C language is belonging to middle level language. C language behaves as a bridge between machine level (low level) languages and high level languages.
 - C language is more user friendly than machine level languages. And, C language does not support all the concepts that high level languages offer. So, C programming language is called as middle level language.
-

22. What do you mean by high level, middle level and low level languages and give an example for each?

High-level languages –

- These level languages provide almost everything that the programmer might need to do as already build into the language.
- Example: Java, Python

Middle-level languages –

- These languages don't provide all the built-in functions found in high level languages, but provide all building blocks that we need to produce the result we want.
- Example: C, C++

Low-level languages –

- These languages provide nothing other than access to the machine's basic instruction set.
 - Example: Assembly language.
-

23. What is the difference between structured oriented, object-oriented and non-structure oriented programming language?

Structured oriented programming language –

- In this type of language, large programs are divided into small programs called functions.
- Prime focus is on functions and procedures that operate on data
- Data moves freely around the systems from one function to another
- Program structure follows "Top Down Approach"
- Example: C, Pascal, ALGOL and Modula-2

Object-oriented programming language –

- In this type of language, programs are divided into objects
- Prime focus is on the data that is being operated and not on the functions or procedures
- Data is hidden and cannot be accessed by external functions
- Program structure follows "Bottom UP Approach"
- Example: C++, JAVA and C# (C sharp)

Non-structure oriented programming language –

- There is no specific structure for programming this language.
 - Example: BASIC, COBOL, FORTRAN
-

24. What is a compiler?

- Compiler is a program that converts human readable code into machine readable code. This process is called compilation.
-

25. What is the difference between assembler, compiler and interpreter?

- Assembler is a program that converts assembly level language (low level language) into machine level language.
- Compiler compiles entire C source code into machine code. Whereas, interpreters converts source code into intermediate code and then this intermediate code is executed line by line.

26. What is meant by protocol?

- The protocol is nothing but a set of rules to be followed by a programmer.

27. List out some of C compilers.

- There are so many compilers available in market for Windows operating system and UNIX. We are listing some of them here for your reference.
 - AMPC
 - CCS C Compiler
 - ch
 - clang
 - Cygwin
 - Digital mars
 - GCC compiler
 - MikroC Compiler
 - Portable C Compiler, Power C, QuickC, Ritchie C Compiler, Small-C

28. Is C language case sensitive?

Yes. C language instructions/commands/functions and everything used in C program are case sensitive.

29. Why is C known as a mother language?

C is known as a mother language because most of the compilers and JVMs are written in C language. Most of the languages which are developed after C language has borrowed heavily from it like C++, Python, Rust, javascript, etc. It introduces new core concepts like arrays, functions, file handling which are used in these languages.

30. Why is C called a mid-level programming language?

C is called a mid-level programming language because it binds the low level and high -level programming language. We can use C language as a System programming to develop the operating system as well as an Application programming to generate menu driven customer driven billing system.

31. Who is the founder of C language?

Dennis Ritchie is the founder of C language.

C, computer programming language developed in the early 1970s by American computer scientist Dennis M. Ritchie at Bell Laboratories (formerly AT&T Bell Laboratories). C was designed as a minimalist language to be used in writing operating systems for minicomputers, such as the DEC PDP 7, which had very limited memories compared with the mainframe computers of the period. The language was devised during 1969–73, alongside the early development of the UNIX operating system. It was based on CPL (Combined Programming Language), which had been first condensed into the B programming language—a stripped-down computer programming language—created in 1969–70 by Ken Thompson, an American computer scientist and a colleague of Ritchie. Ritchie subsequently rewrote and restored features from CPL to create C and eventually rewrote the UNIX operating system in the new language.

32. What are the features of the C language?

The main features of C language are given below:

- **Simple:** C is a simple language because it follows the structured approach, i.e., a program is broken into parts
- **Portable:** C is highly portable means that once the program is written can be run on any machine with little or no modifications.
- **Mid Level:** C is a mid-level programming language as it combines the low- level language with the features of the high-level language.
- **Structured:** C is a structured language as the C program is broken into parts.
- **Fast Speed:** C language is very fast as it uses a powerful set of data types and operators.
- **Memory Management:** C provides an inbuilt memory function that saves the memory and improves the efficiency of our program.
- **Extensible:** C is an extensible language as it can adopt new features in the future.

33. What is command line argument?

The argument passed to the main() function while executing the program is known as command line argument. For example:

```
main(int count, char *args[]){  
    //code to be executed  
}
```

34. What is the acronym for ANSI?

The ANSI stands for " American National Standard Institute." It is an organization that maintains the broad range of disciplines including photographic film, computer languages, data encoding, mechanical parts, safety and more.

35. Who is the main contributor in designing the C language after Dennis Ritchie?

Brian Kernighan.

Brian Wilson Kernighan is a Canadian computer scientist. He worked at Bell Labs and contributed to the development of Unix alongside Unix creators Ken Thompson and Dennis Ritchie. Kernighan's name became widely known through co-authorship of the first book on the C programming language with Dennis Ritchie.

Born: 1 January 1942 (age 80 years), Toronto, Canada

Nationality: Canadian

36. What is a token?

A C program consists of various tokens and a token is either a keyword, an identifier, a constant, a string literal, or a symbol.

37. What is the purpose of the keyword typedef?

It is used to alias the existing type. Also used to simplify the complex declaration of the type.

38. What is difference between including the header file with-in angular braces < > and double quotes " "

If a header file is included with in < > then the compiler searches for the particular header file only with in the built in include path. If a header file is included with in " ", then the compiler searches for the particular header file first in the current working directory, if not found then in the built in include path.

39. What is keyword auto for?

By default every local variable of the function is automatic (auto). In the below function both the variables 'i' and 'j' are automatic variables.

```
void f() {  
    int i;  
    auto int j;  
}
```

NOTE - A **global** variable can't be an automatic variable.

40. What is volatile keyword?

The volatile keyword is intended to prevent the compiler from applying any optimizations on objects that can change in ways that cannot be determined by the compiler. Objects declared as volatile are omitted from optimization

because their values can be changed by code outside the scope of current code at any time. See Understanding "volatile" qualifier in C for more details.

41. What are main characteristics of C language?

C is a procedural language. The main features of C language include low-level access to memory, simple set of keywords, and clean style. These features make it suitable for system programming like operating system or compiler development.

42. How will you print "Hello World" without semicolon?

```
int main(void)
{
    if (printf("Hello World"));
}
```

43. What is a static identifier?

A file-scope variable that is declared static is visible only to functions within that file. A function-scope or block-scope variable that is declared as static is visible only within that scope. Furthermore, static variables only have a single instance. In the case of function- or block-scope variables, this means that the variable is not "automatic" and thus retains its value across function invocations.

44. What are the valid places for the keyword break to appear.

Break can appear only with in the looping control and switch statement. The purpose of the break is to bring the control out from the said blocks.

45. Describe the history of C programming language.

1960	• ALGOL	International Group
1967	• BCPL	Martin Richards
1970	• B	Ken Thompson
1972	• Traditional C	Dennis Ritchie
1978	• K&R C	Brain Kernighan and Dennis Ritchie
1989	• ANSI C	ANSI Committee
1990	• ANSI/ISO C	ISO Committee

- C programming language is a structure oriented programming language, was developed at Bell Laboratories in 1972 by Dennis Ritchie
- C language features were derived from earlier language called "B" (Basic Combined Programming Language – BCPL)
- In earlier days, programs were written in assembly level language. So, it had happened to write very big programs to perform specific tasks using assembly code.
- But, 'B' language could perform the same task in few lines of program and it was faster than assemble language code.

- But, B language did not support some features like data types and structures etc. So, this was a drawback of B language. So, Dennis Ritchie developed C language by keeping most part of the B language and adding many features that produced powerful and effective outputs.
- So, C language was invented for implementing UNIX operating system. Most of the UNIX components were rewritten in C.
- In 1978, Dennis Ritchie and Brian Kernighan published the first edition "The C Programming Language" and commonly known as K&R C
- In 1983, the American National Standards Institute (ANSI) established a committee to provide a modern, comprehensive definition of C. The resulting definition, the ANSI standard, or "ANSI C", was completed late 1988.
- C89/C90 standard – First standardized specification for C language was developed by American National Standards Institute in 1989. C89 and C90 standards refer to the same programming language.
- C99 standard – Next revision was published in 1999 that introduced new futures like advanced data types and other changes.
- C11 standard adds new features to C and library like type generic macros, anonymous structures, improved Unicode support, atomic operations, multi-threading, and bounds-checked functions. It also makes some portions of the existing C99 library optional, and improves compatibility with C++.
- Embedded C includes features not available in normal C like fixed-point arithmetic, named address spaces, and basic I/O hardware addressing.

[learn more](#)

46. Which key word is used to perform unconditional branching?

goto

47. Which compiler switch to be used for compiling the programs using math library with gcc compiler?

Option -lm to be used as > gcc -lm

48. Can the curly brackets { } be used to enclose a single line of code?

While curly brackets are mainly used to group several lines of codes, it will still work without error if you used it for a single line. Some programmers prefer this method as a way of organizing codes to make it look clearer, especially in conditional statements.

49. What are header files and what are its uses in C programming?

Header files are also known as library files. They contain two essential things: the definitions and prototypes of functions being used in a program. Simply put, commands that you use in C programming are actually functions that are defined from within each header files. Each header file contains a set of functions. For example: stdio.h is a header file that contains definition and prototypes of commands like printf and scanf.

50. What is syntax error?

Syntax errors are associated with mistakes in the use of a programming language. It maybe a command that was misspelled or a command that must was entered in lowercase mode but was instead entered with an upper case character. A misplaced symbol, or lack of symbol, somewhere within a line of code can also lead to syntax error.

51. Can two or more operators such as and be combined in a single line of program code?

Yes, it's perfectly valid to combine operators, especially if the need arises. For example: you can have a code like `" printf ("Hello 'World'") "` to output the text "Hello" on the first line and "World" enclosed in single quotes to appear on the next two lines.

52. Why is it that not all header files are declared in every C program?

The choice of declaring a header file at the top of each C program would depend on what commands/functions you will be using in that program. Since each header file contains different function definitions and prototype, you would be using only those header files that would contain the functions you will need. Declaring all header files in every program would only increase the overall file size and load of the program, and is not considered a good programming style.

53. When is the "void" keyword used in a function?

When declaring functions, you will decide whether that function would be returning a value or not. If that function will not return a value, such as when the purpose of a function is to display some outputs on the screen, then "void" is to be placed at the leftmost part of the function header. When a return value is expected after the function execution, the data type of the return value is placed instead of "void".

54. What are compound statements?

Compound statements are made up of two or more program statements that are executed together. This usually occurs while handling conditions wherein a series of statements are executed when a TRUE or FALSE is evaluated. Compound statements can also be executed within a loop. Curly brackets { } are placed before and after compound statements.

55. What is wrong in this statement?

```
scanf("%d",whatnumber);
```

An ampersand & symbol must be placed before the variable name whatnumber. Placing & means whatever integer value is entered by the user is stored at the "address" of the variable name. This is a common mistake for programmers, often leading to logical errors.

56. What are comments and how do you insert it in a C program?

Comments are a great way to put some remarks or description in a program. It can serve as a reminder on what the program is all about, or a description on why a certain code or function was placed there in the first place. Comments begin with `/*` and ended by `*/` characters. Comments can be a single line, or can even span several lines. It can be placed anywhere in the program.

57. What are reserved words?

Reserved words are words that are part of the standard C language library. This means that reserved words have special meaning and therefore cannot be used for purposes other than what it is originally intended for. Examples of reserved words are `int`, `void`, and `return`.

58. Not all reserved words are written in lowercase. TRUE or FALSE?

FALSE. All reserved words must be written in lowercase; otherwise the C compiler would interpret this as unidentified and invalid

59. What are run-time errors?

These are errors that occur while the program is being executed. One common instance wherein run-time errors can happen is when you are trying to divide a number by zero. When run-time errors occur, program execution will pause, showing which program line caused the error.

60. What is the difference between functions `abs()` and `fabs()`?

These 2 functions basically perform the same action, which is to get the absolute value of the given value. `Abs()` is used for integer values, while `fabs()` is used for floating type numbers. Also, the prototype for `abs()` is under `<math.h>`, while `fabs()` is under `<math.h>`.

61. What is the general form of a C program?

A C program begins with the preprocessor directives, in which the programmer would specify which header file and what constants (if any) to be used. This is followed by the main function heading. Within the main function lies the variable declaration and program statement.

62. What is the use of a semicolon (;) at the end of every program statement?

It has to do with the parsing process and compilation of the code. A semicolon acts as a delimiter, so that the compiler knows where each statement ends, and can proceed to divide the statement into smaller elements for syntax checking.

63. What is Standard Header Files ?

Prototypes of the library functions are given in several standard header files. For example, `stdio.h` contains prototypes for `printf`, `scanf`, `putchar` and `getchar`.

Other standard header files are:

1. `assert.h` assertions
2. `ctype.h` character class tests
3. `float.h` system limits for floating point types

4. limits.h system limits for integral types
5. math.h mathematical functions
6. setjmp.h non-local jumps
7. signal.h signals and error handling
stdarg.h variable length parameter lists
8. stdlib.h utility functions; number conversions, memory allocation, exit and system , Quick Sort
9. string.h string functions
10. time.h date and time functions

To include these standard header files in your code, use the preprocessor directive **#include** and place angle brackets around the name of the file, e.g.

```
#include<math.h>
```

64. What is printf()?

- printf() is an inbuilt library function in C which is available in C library by default. This function is declared and related macros are defined in "stdio.h" header file.
- printf() function is used to print the "character, string, float, integer, octal and hexadecimal values" onto the output screen.

65. What is scanf()?

- scanf() function is an inbuilt library function in C which is available in C library by default. This function is declared and related macros are defined in "stdio.h" header file.
- scanf() function is used to read character, string, numeric data from keyboard.

66. Execution of a C program starts from which function?

Always, execution of a C program starts from main() function.

67. What are all the sections that a C program may have and must have?

There are many sections in a C program structure. They are,

- Documentation section
- Link Section
- Definition Section
- Global declaration section
- Function prototype declaration section
- Main function
- User defined function section

main() function section is the important section in a C program as program execution starts from main() function only in C language. A C program may not have all other sections except main() function.

68. What is IDE?

- IDE is nothing but Integrated Development Environment. IDE is a tool that provides user interface with compilers to create, compile and execute C programs.
- Example: Turbo C++, Borland C++ and DevC++. These provide Integrated Development Environment with compiler for both C and C++ programming language.

69. What is header file in C language?

- Header file is a file that contains function declaration and macro definition for C in-built library functions.
- All C standard library functions are declared in many header files which are saved as file_name.h.

- We are including these header files in our C program using “#include <file_name.h>” command to make use of the functions those are declared in the header files.
 - When we include header files in our C program using “#include <filename.h>” command, all C code of the header files are included in C program. Then, this C program is compiled by compiler and executed.
-

70. What is void in C?

- Void is an empty data type that has no value.
- We use void data type in functions when we don't want to return any value to the calling function.
- Example:

`void sum (int a, int b);` – This function won't return any value to the calling function.

`int sum (int a, int b);` – This function will return value to the calling function.

- We use void data type in pointer like “void *p”. It means, pointer “p” is neither pointing to int data type nor char data type. It acts as generic pointer. We are using void pointer when we are not sure on the data type that this pointer will point to. We can use void pointer to refer either integer data or char data. But, this void pointer should not be dereferenced without explicit type casting.
 - We use void in functions as “int function_name (void)”. Here void means, this function does not pass any argument.
-

71. What is token in C?

- C tokens are the basic buildings blocks in C language which are constructed together to write a C program.
 - Each and every smallest individual unit in a C program are known as C tokens.
-

72. What are the types of C tokens?

- C tokens are of six types. They are,
 1. Keywords (eg: int, while),
 2. Identifiers (eg: main, total),
 3. Constants (eg: 10, 20),
 4. Strings (eg: "total", "hello"),
 5. Special symbols (eg: (), {}),
 6. Operators (eg: +, /, -, *)

73. What is identifier in C?

- Each program elements in a C program are given a name called identifiers.
- Names given to identify Variables, functions and arrays are examples for identifiers.
- Example: x is a name given to integer variable in above program

74. What is keyword in C?

- Keywords are pre-defined words in a C compiler. Each keyword is meant to perform a specific function in a C program.
- Since keywords are referred names for compiler, they can't be used as variable name.

[For More details visit this tutorial:](#)

<https://www.atnyla.com/tutorial/keywords-in-c/1/163>

75. List out some keywords available in C language.

- Below are some of keywords that C language offers.
 - auto, double, int, struct, break, else, long, switch, case, enum, register, typedef, char, extern, return, union, const, float, short, unsigned, continue, for, signed, void, default, goto, sizeof, volatile, do, , if, static, while
-

76. What is the syntax for comments in C?

Below is the syntax for comments in C. The characters or words or anything which are given between `"/**"` and `"/**"`, won't be considered by C compiler for compilation process. These will be ignored by C compiler during compilation.

Syntax: `/* your comments here */`

77. What is the use of `printf()` and `scanf()` functions?

printf(): The `printf()` function is used to print the integer, character, float and string values on to the screen.

Following are the format specifier:

- **%d:** It is a format specifier used to print an integer value.
- **%s:** It is a format specifier used to print a string.
- **%c:** It is a format specifier used to display a character value.
- **%f:** It is a format specifier used to display a floating point value.

scanf(): The `scanf()` function is used to take input from the user.

78. Can we compile a program without `main()` function?

Yes, we can compile, but it can't be executed.

But, if we use `#define`, we can compile and run a C program without using the `main()` function. For example:

```
#include<stdio.h>
#define start main
void start() {
    printf("Hello");
}
```

79. What is a token?

The Token is an identifier. It can be constant, keyword, string literal, etc. A token is the smallest individual unit in a program. C has the following tokens:

1. Identifiers: Identifiers refer to the name of the variables.
2. Keywords: Keywords are the predefined words that are explained by the compiler.
3. Constants: Constants are the fixed values that cannot be changed during the execution of a program.
4. Operators: An operator is a symbol that performs the particular operation.
5. Special characters: All the characters except alphabets and digits are treated as special characters.

80. What is the difference between `getch()` and `getche()`?

The **`getch()`** function reads a single character from the keyboard. It doesn't use any buffer, so entered data will not be displayed on the output screen.

The **getche()** function reads a single character from the keyboard, but data is displayed on the output screen. Press Alt+f5 to see the entered character.

Let's see a simple example

```
#include<stdio.h>
#include<conio.h>
int main()
{

    char ch;
    printf("Enter a character ");
    ch=getch(); // taking an user input without printing the value.
    printf("\nvalue of ch is %c",ch);
    printf("\nEnter a character again ");
    ch=getche(); // taking an user input and then displaying it on the
    screen.
    printf("\nvalue of ch is %c",ch);
    return 0;
}
```

Output:

```
Enter a character
value of ch is a
Enter a character again a
value of ch is a
```

In the above example, the value entered through a `getch()` function is not displayed on the screen while the value entered through a `getche()` function is displayed on the screen.

81. What is the newline escape sequence?

The new line escape sequence is represented by "`\n`". It inserts a new line on the output screen.

An escape sequence in C language is a sequence of characters that doesn't represent itself when used inside string literal or character.

It is composed of two or more characters starting with backslash `\`. For example: `\n` represents new line.

List of Escape Sequences in C

Escape Sequence	Meaning
<code>\a</code>	Alarm or Beep
<code>\b</code>	Backspace
<code>\f</code>	Form Feed
<code>\n</code>	New Line
<code>\r</code>	Carriage Return
<code>\t</code>	Tab (Horizontal)
<code>\v</code>	Vertical Tab
<code>\\</code>	Backslash
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote
<code>\?</code>	Question Mark
<code>\nnn</code>	octal number
<code>\xhh</code>	hexadecimal number
<code>\0</code>	Null

82. What is the maximum length of an identifier?

It is 32 characters ideally but implementation specific.

83. Write a program to print "hello world" without using a semicolon?

```
#include<stdio.h>
void main(){
    if (printf("hello world")){} // It prints the ?hello world? on the
    screen.
}
```

84. How a negative integer is stored.

Get the two's complement of the same positive integer. Eg: 1011 (-5)

Step-A - One's complement of 5 : 1010

Step-B - Add 1 to above, giving 1011, which is -5

85. Can I use "int" data type to store the value 32768? Why?

No. "int" data type is capable of storing values from -32768 to 32767. To store 32768, you can use "long int" instead. You can also use "unsigned int", assuming you don't intend to store negative values.

86. What are the different data types in C?

The basic data types are int, char, and float. Int is used to declare variables that will be storing integer values. Float is used to store real numbers. Char can store individual character values.

87. What is data type in C?

- Data types in C language are defined as the data storage format that a variable can store a data to perform a specific operation.

- Data types are used to define a variable before to use in a program.
 - Size of variable, constant and array are determined by data types.
-

88. What is the difference between int, char, float and double data types?

- Integer data type allows a variable to store numeric values. The storage size of int data type is 2 or 4 or 8 byte. It varies depend upon the processor in the CPU.
 - Character data type allows a variable to store only one character. Storage size of character data type is 1.
 - Float data type allows a variable to store decimal values. Storage size of float data type is 4. This also varies depend upon the processor in the CP.
 - Double data type is also same as float data type which allows up-to 10 digits after decimal.
-

89. What is modifier in C?

- The amount of memory space to be allocated for a variable is derived by modifiers.
 - Modifiers are prefixed with basic data types to modify (either increase or decrease) the amount of storage space allocated to a variable.
 - For example, storage space for int data type is 4 byte for 32 bit processor. We can increase the range by using long int which is 8 byte. We can decrease the range by using short int which is 2 byte.
-

90. What are different types of modifiers in C?

- There are 5 modifiers available in C language. They are,
 - short
 - long
 - signed

- unsigned
- long long

91. What is typecasting?

The typecasting is a process of converting one data type into another is known as typecasting. If we want to store the floating type value to an int type, then we will convert the data type into another data type explicitly.

```
(type_name) expression;
```

92. What is the difference between variable declaration and variable definition? (in a line)

Declaration associates type to the variable whereas definition gives the value to the variable.

93. Where an automatic variable is stored?

Every local variable by default being an auto variable is stored in stack memory.

94. When should we use the register storage specifier?

If a variable is used most frequently then it should be declared using register storage specifier, then possibly the compiler gives CPU register for its storage to speed up the look up of the variable.

95. What is the purpose of extern storage specifier?

Used to resolve the scope of global symbol.

```
Example:  
main() {  
    extern int j;  
    Printf("%d",j);  
}
```

```
int j = 20;
```

96. What is a static variable?

A static local variables retains its value between the function call and the default value is 0. The following function will print 1 2 3 if called thrice.

```
void f() {  
    static int i;  
    ++i;  
    printf("%d ",i);  
}
```

If a global variable is static then its visibility is limited to the same source code.

97. Can a variable be both const and volatile?

yes, the const means that the variable cannot be assigned a new value. The value can be changed by other code or pointer. For example the following program works fine.

```
int main(void)
{
    const volatile int local = 10;
    int *ptr = (int*) &local;
    printf("Initial value of local : %d", local);
    *ptr = 100;
    printf("Modified value of local: %d", local);
    return 0;
}
```

98. What are local static variables? What is their use?

A local static variable is a variable whose lifetime doesn't end with a function call where it is declared. It extends for the lifetime of complete program. All calls to the function share the same copy of local static variables. Static variables can be used to count the number of times a function is called. Also, static

variables get the default value as 0. For example, the following program prints "0 1"

```
#include
void fun()
{
    // static variables get the default value as 0.
    static int x;
    printf("%d ", x);
    x = x + 1;
}

int main()
{
    fun();
    fun();
    return 0;
}
// Output: 0 1
```

99. What is the difference between declaration and definition of a variable/function

Declaration of a variable/function simply declares that the variable/function exists somewhere in the program but the memory is not allocated for them. But the declaration of a variable/function serves an important role. And that is the type of the variable/function. Therefore, when a variable is declared, the program knows the data type of that variable. In case of function declaration, the program knows what are the arguments to that functions, their data types,

the order of arguments and the return type of the function. So that's all about declaration. Coming to the definition, when we define a variable/function, apart from the role of declaration, it also allocates memory for that variable/function. Therefore, we can think of definition as a super set of declaration. (or declaration as a subset of definition). From this explanation, it should be obvious that a variable/function can be declared any number of times but it can be defined only once. (Remember the basic principle that you can't have two locations of the same variable/function).

```
// This is only declaration. y is not allocated memory by this statement
extern int y;

// This is both declaration and definition, memory to x is allocated by
this statement.
int x;
```

100. What are different storage class specifiers in C?

auto, register, static, extern

101. What is the difference between declaration and definition of a variable/function

Declaration of a variable/function simply declares that the variable/function exists somewhere in the program but the memory is not allocated for them. But

the declaration of a variable/function serves an important role. And that is the type of the variable/function. Therefore, when a variable is declared, the program knows the data type of that variable. In case of function declaration, the program knows what are the arguments to that functions, their data types, the order of arguments and the return type of the function. So that's all about declaration. Coming to the definition, when we define a variable/function, apart from the role of declaration, it also allocates memory for that variable/function. Therefore, we can think of definition as a super set of declaration. (or declaration as a subset of definition). From this explanation, it should be obvious that a variable/function can be declared any number of times but it can be defined only once. (Remember the basic principle that you can't have two locations of the same variable/function).

```
// This is only declaration. y is not allocated memory by this statement
extern int y;

// This is both declaration and definition, memory to x is allocated by
this statement.
int x;
```

102. What does static variable mean?

Static variables are the variables which retain their values between the function calls. They are initialized only once their scope is within the function in which they are defined.

103. Where is the auto variables stored?

Auto variables can be stored anywhere, so long as recursion works. Practically, they're stored on the stack. It is not necessary that always a stack exists. You could theoretically allocate function invocation records from the heap.

104. Where does global, static, and local, register variables, free memory and C Program instructions get stored?

Global: Wherever the linker puts them. Typically the .BSS segment? on many platforms.

Static: Again, wherever the linker puts them. Often, they're intermixed with the globals. The only difference between globals and statics is whether the linker will resolve the symbols across compilation units.

Local: Typically on the stack, unless the variable gets register allocated and never spills.

Register: Nowadays, these are equivalent to .Local? variables. They live on the stack unless they get register-allocated.

105. Describe about storage allocation and scope of global, extern, static, local and register variables?

Globals have application-scope. They are available in any compilation unit that includes an appropriate declaration (usually brought from a header file). They're stored wherever the linker puts them, usually a place called the BSS segment.

Extern: This is essentially "global".

Static: Stored the same place as globals, typically, but only available to the compilation unit that contains them. If they are block-scope global, only available within that block and its subblocks.

Local: Stored on the stack, typically. Only available in that block and its subblocks. (Although pointers to locals can be passed to functions invoked from within a scope where that local is valid.)

Register: See tirade above on "local" vs. "register". The only difference is

that the C compiler will not let you take the address of something you've declared as "register".

106. What are register variables? What are the advantages of using register variables?

If a variable is declared with a register storage class, it is known as register variable. The register variable is stored in the CPU register instead of main memory. Frequently used variables are declared as register variable as its access time is faster.

107. What is storage class? What are the different storage classes in C?

The storage class is an attribute that changes the behavior of a variable. It controls the lifetime, scope and linkage. The storage classes in C are auto, register, and extern, static, typedef.

108. What is variable initialization and why is it important?

This refers to the process wherein a variable is assigned an initial value before it is used in the program. Without initialization, a variable would have an unknown value, which can lead to unpredictable outputs when used in computations or other operations.

109. What is wrong with this program statement? `void = 10;`

The word void is a reserved word in C language. You cannot use reserved words as a user-defined variable.

110. Is this program statement valid? `INT = 10.50;`

Assuming that INT is a variable of type float, this statement is valid. One may think that INT is a reserved word and must not be used for other purposes. However, recall that reserved words are express in lowercase, so the C compiler will not interpret this as a reserved word.

111. What are global variables and how do you declare them?

Global variables are variables that can be accessed and manipulated anywhere in the program. To make a variable global, place the variable declaration on the upper portion of the program, just after the preprocessor directives section.

112. What is Blocks and Scope in c ?

C is a block-structured language. Blocks are delimited by { and}. Every block can have its own local variables. Blocks can be defined wherever a C statement could be used. No semi-colon is required after the closing brace of a block.

Code:

```
#include<stdio.h>
#define semicolon ;
void main()
{

    int a = 5;
    printf("\n%d", a);

    {
        int a = 2;
        printf("\n%d", a);
    }
}
```

```
}
```

Output:

```
5
```

```
2Press any key to continue . . .
```

Reference to a variable will be to the variable of that name in the nearest enclosing block.

113. What is Variable Storage Classes?

auto: The default class. Automatic variables are local to their block. Their storage space is reclaimed on exit from the block.

register: If possible, the variable will be stored in a processor register. May give faster access to the variable. If register storage is not possible, then the variable will be of automatic class. Use of the register class is not recommended, as the compiler should be able to make a better judgment about which variables to hold in registers, in fact the injudicious use of register variables may slow down the program.

static: On exit from the block, static variables are not reclaimed. They keep their value. On re-entry to the block, the variable will have its old value.

extern: Allows access to external variables. An external variable is either a global variable or a variable defined in another source file. External variables

are

defined outside of any function. (Note: Variables passed to a function and modified by way of a pointer are not external variables)

static external: External variables can be accessed by any function in any source file which make up the final program. Static external variables can only be accessed by functions in the same file as the variable declaration.

114. What is Variable Declaration ?

Variables should be declared either outside a function or at the start of a block of code, after the opening { and before any other statements.

```
int miles, yards; /* global variables */
main()
{
float kilometres; /* local variables */
```

115. What is variable in C?

- C variable is a named location in a memory where a program can manipulate the data. This location is used to hold the value of the variable.
- The value of the C variable may get change in the program.
- C variable might be belonging to any of the data type like int, float, char etc.

116. What is the difference between constant and variable in C?

Constant values can't be modified by the program once it is declared. But, variable values can be modified by the program.

117. Can variable name start with numbers?

No. Variable names can't start with numbers as per the variable naming rule.

118. What is the difference between variable declaration and variable definition in C?

- Variable declaration tells the compiler about data type and size of the variable. Whereas, variable definition allocates memory to the variable
- Variable can be declared many times in a program. But, definition can happen only one time for a variable in a program.
- Variable declaration is for assignment of properties and identification to a variable. Whereas, variable definition is for assignments of storage space to a variable

119. What are the different types of variable in C?

- Local variable
- Global variable
- Environment variable

120. What is local variable in C?

- The variables which are having scope/life only within the function are called local variables.
- These variables are declared within the function and can't be accessed outside the function.

For **Full version book** visit our website atnyla.com and order the book using official email address